

Terminal

Architecture, Data Sources & Open-Source Security Guide

Created by Oscar Zarraga Perez

A technical reference describing how the project collects, caches, and serves market data — written so the source code can be released publicly without exposing credentials or personal information.

Copyright © 2026 Oscar Zarraga Perez · All rights reserved

Document Version: 3.0 — released without warranty.

Disclaimer & Limitation of Liability

No warranty. The Terminal project is provided "AS IS" and "AS AVAILABLE", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, accuracy, completeness, non-infringement, or continued availability of any upstream data source. The project is a personal-use, educational software experiment. Nothing in this document or in the source code creates a contract, an engagement, or a service-level commitment of any kind.

Not investment advice. The data displayed by the project is aggregated from third-party public sources whose accuracy, timeliness, and continuity cannot be guaranteed. The project is not a broker-dealer, investment adviser, financial planner, accountant, auditor, or tax adviser. None of the output of the project, of this document, or of the source code constitutes investment, legal, accounting, tax, or financial advice. Any decision to buy, sell, hold, or evaluate a security is the sole responsibility of the person making that decision and should be taken only after consulting an appropriately licensed professional.

Limitation of liability. To the maximum extent permitted by applicable law, the authors and contributors of the project disclaim all liability for any direct, indirect, incidental, special, consequential, exemplary, or punitive damages — including but not limited to lost profits, lost data, trading losses, missed opportunities, regulatory fines, reputational harm, or any other commercial damage — arising out of or in connection with the use, misuse, or inability to use the project, the documentation, the source code, or any data obtained through the project, even if advised of the possibility of such damages.

Operator responsibility. Anyone who downloads, runs, modifies, redistributes, or deploys the project (the "operator") is solely responsible for: (a) compliance with the terms of service of every upstream data source the project queries; (b) compliance with all applicable laws, including securities, market-data, privacy, export, and tax laws in the operator's jurisdiction; (c) the security of any API keys, credentials, or other secrets the operator chooses to configure; (d) any consequences arising from public, commercial, or multi-user deployments of the project.

Third-party services. The project queries third-party endpoints. It does not control, endorse, or warrant any third-party content, service availability, accuracy, or terms. Endpoints may change, rate-limit, or cease without notice. Trademarks, service marks, and logos referenced in this document are the property of their respective owners; references are nominative and do not imply endorsement, affiliation, or sponsorship in either direction.

By using the project in any form, the operator acknowledges and accepts these terms in full.

Setup & Quick-Start Guide

How to install and run the Terminal on macOS, Linux, or Windows. No build step, no *pip install*, no third-party Python packages — the entire backend runs on the Python standard library.

Companion to *Terminal Architecture.pdf*. Version 3.0 — Copyright © 2026 Oscar Zarraga Perez.

1. Prerequisites

Python 3.9 or newer. Nothing else is required. The UI runs in your default browser at **`http://127.0.0.1:8787/`** once the launcher script starts the server.

Check your Python version:

```
python3 --version
```

macOS and modern Linux distributions usually ship Python 3 by default. On Windows, install from python.org/downloads and tick "**Add Python to PATH**" during installation.

2. Install

There is nothing to compile or install. Download the ZIP (or clone the repository) and leave the folder structure intact. Everything you need ships in the same directory.

```
git clone https://github.com/&lt;your-fork&gt;/unified-terminal.git cd
unified-terminal
```

The folder should contain *markets_data_api.py*, the four HTML pages (*terminal.html*, *markets.html*, *calendar.html*, *policy.html*), the three launcher scripts (*start_mac_linux.command*, *start_windows.bat*, *start_windows.vbs*), *keys.json.example*, and the documentation PDFs.

3. Run it

macOS & Linux

Double-click *start_mac_linux.command*. The first time, macOS Gatekeeper may require you to right-click → Open. Or run from a terminal:

```
./start_mac_linux.command
```

Windows

Two options:

start_windows.vbs — recommended. Launches the Python server with **no visible console window**; runs silently in the background. To stop the server, end *python.exe* in Task Manager.

start_windows.bat — delegates to the .vbs launcher and exits, so a cmd window flashes briefly then closes. Same result either way.

Any shell, manually

```
python3 markets_data_api.py --port 8787
```

All three launch the same backend on port 8787. Open **http://127.0.0.1:8787/** in your browser if it does not open automatically.

4. Optional: free-tier API keys

The terminal works fully without any API keys — it uses Yahoo Finance, Stooq, SEC EDGAR, CoinGecko, Binance, mempool.space, MarketWatch, CNBC's free quote API, frankfurter.app FX rates, Nasdaq's public calendar, the Federal Register API, and a dozen public RSS feeds. Adding the four optional keys below expands data depth: richer company fundamentals, more reliable real-time quotes when Yahoo throttles, additional news sources, and a complete index/FX fallback chain.

Sign-up links (all free):

Provider	Free tier	Sign-up
Finnhub	60 req/min	finnhub.io/register
Financial Modeling Prep	250 req/day	site.financialmodelingprep.com/developer/docs
Twelve Data	8 req/min, 800/day	twelvedata.com/register
Alpha Vantage	5 req/min, 500/day	alphavantage.co/support/#api-key

Once you have registered, copy the example file:

```
cp keys.json.example keys.json
```

Edit `keys.json` and paste your keys into the matching fields. Leave any field blank if you don't have that key — the provider will simply be skipped. Save the file. **Keys are auto-reloaded** within 2 seconds — no server restart required.

Security note: `keys.json` is excluded by the shipped `.gitignore`. Never commit it to a public repo. Each key is independently optional; the terminal degrades gracefully when any are missing.

5. Customize the data you see

Watchlist

In the Markets tab, the left WATCHLIST panel has an ADD TICKER input. Type a ticker (AAPL, BTC-USD, EURUSD=X, ^GSPC) and press Enter, or click the × on a row to remove it. The list persists in localStorage under `terminal_wl` and is shared with the Watchlist Feed tab.

Indices, FX, Crypto side panels

Each of the three side panels in the Markets tab is editable. Each has its own ADD input at the top (with format hints) and an × on every row. CLR in the panel header empties the list. Defaults are restored if you have never customized:

Panel	Format	Examples
INDICES	Yahoo index symbol	^GSPC, ^DJI, ^IXIC, ^FTSE, ^N225
FX	Yahoo FX pair ending in =X	EURUSD=X, USDJPY=X, GBPUSD=X
CRYPTO	CoinGecko ticker ending in -USD	BTC-USD, ETH-USD, SOL-USD

Tabs and tab visibility

When you load a non-equity symbol, the bottom tab bar adapts: indices, FX, and crypto show only the NEWS tab; futures show OVERVIEW + NEWS; stocks show every tab (KEY STATS, INCOME, BALANCE, CASH FLOW, EARNINGS, NEWS, PRESS REL, PROFILE, SEC FILINGS). The COMPANY block on the right shows the instrument's name, class, currency, and exchange for non-stocks instead of the equity company sheet.

Policy Watch — date controls and views

The Policy Watch tab has seven sub-views: **All**, **Committee Hearings**, **Federal Register**, **Regulators** (SEC/CFTC/Fed/Treasury), **Central Banks (Intl)**, **BTC + ETH ETF Flows**, and **All Sources**. The All Sources view is a clickable directory of 54 curated regulatory / macro / on-chain / ETF / international links — every one opens as an in-app tab through the proxy. The REFRESH button at the top-right reloads everything on demand; otherwise the page auto-refreshes every 10 minutes.

6. Troubleshooting

"Backend not running" banner overlays the page

Either the Python server is not running, or it is running on a different port. Confirm by visiting <http://127.0.0.1:8787/api/healthz> — it should return `{"ok":true,...}`. If not, restart with the launcher script. On Windows, double-click `start_windows.vbs` (silent launch).

Watchlist prices are blank or all the same number

Yahoo has rate-limited your IP. The terminal falls back through a chain of independent providers: **Finnhub** → **FMP** → **Twelve Data** → **Stooq** → **MarketWatch (indices)** → **CNBC (indices)** → **Binance (crypto)** → **frankfurter.app (FX)**. Add at least a Finnhub key — the free 60 req/min is plenty for an 8-ticker watchlist refreshing every 30 seconds. A multi-pass garbage-data detector inside the Yahoo helper rejects responses where more than 50% of tickers share an exact price, where $\geq 40\%$ cluster within 1% of the median, or where any single row's price differs from its previous close by more than 50% — so degraded data never reaches the UI.

SEC filings show only the policy banner, no content

SEC.gov rejects generic User-Agents. The default `SEC_USER_AGENT` is *Unified-Terminal contact@example.com* — for production, set it to an address you actually own (SEC's fair-access policy requires this):

```
export SEC_USER_AGENT="My Company me@mycompany.com" python3 markets_data_api.py
```

The terminal also intercepts SEC EDGAR landing URLs (`cgi-bin/browse-edgar?CIK=...`) and renders a custom filings page from data.sec.gov/submissions so the modern SEC SPA doesn't fail inside the proxied iframe.

Indices / FX / Crypto side panels are empty

These instruments now use a dedicated multi-source chain: **indices** via MarketWatch HTML and CNBC's quote API (both no-key, parallel); **FX** via frankfurter.app's USD-based rate table (no key) plus Twelve Data; **crypto** via CoinGecko followed by Binance (no key). The chain runs in parallel inside `resolve_prices()` and stops as soon as a price returns. Click REFRESH on the Markets tab title bar to force a re-fetch.

Policy Watch tabs show empty rows

Most Policy sources are RSS feeds with a 15-minute server-side cache. On first load they populate within about 5 seconds. If they stay empty after a minute, your machine probably cannot reach the source domains (*federalregister.gov*, *sec.gov*, *federalreserve.gov*, etc.). Corporate firewalls sometimes block these.

News clicks show Google's "Redirect Notice"

Google News article URLs (news.google.com/articles/CBM...) embed the publisher's URL in a URL-safe base64 token. The terminal's `_decode_google_news_url()` helper extracts the publisher URL at fetch time, so most headlines load the real article (Bloomberg, Reuters, MarketWatch, etc.) directly. For tokens that use the newer server-side-ID format, the `_mustEscapeIframe()` check in `terminal.html`

hands the URL to your real browser tab so the redirect resolves naturally.

Port 8787 already in use

Launch on a different port:

```
python3 markets_data_api.py --port 9000
```

© 2026 Oscar Zarraga Perez · MIT License · See *Terminal Architecture.pdf* for architecture details.